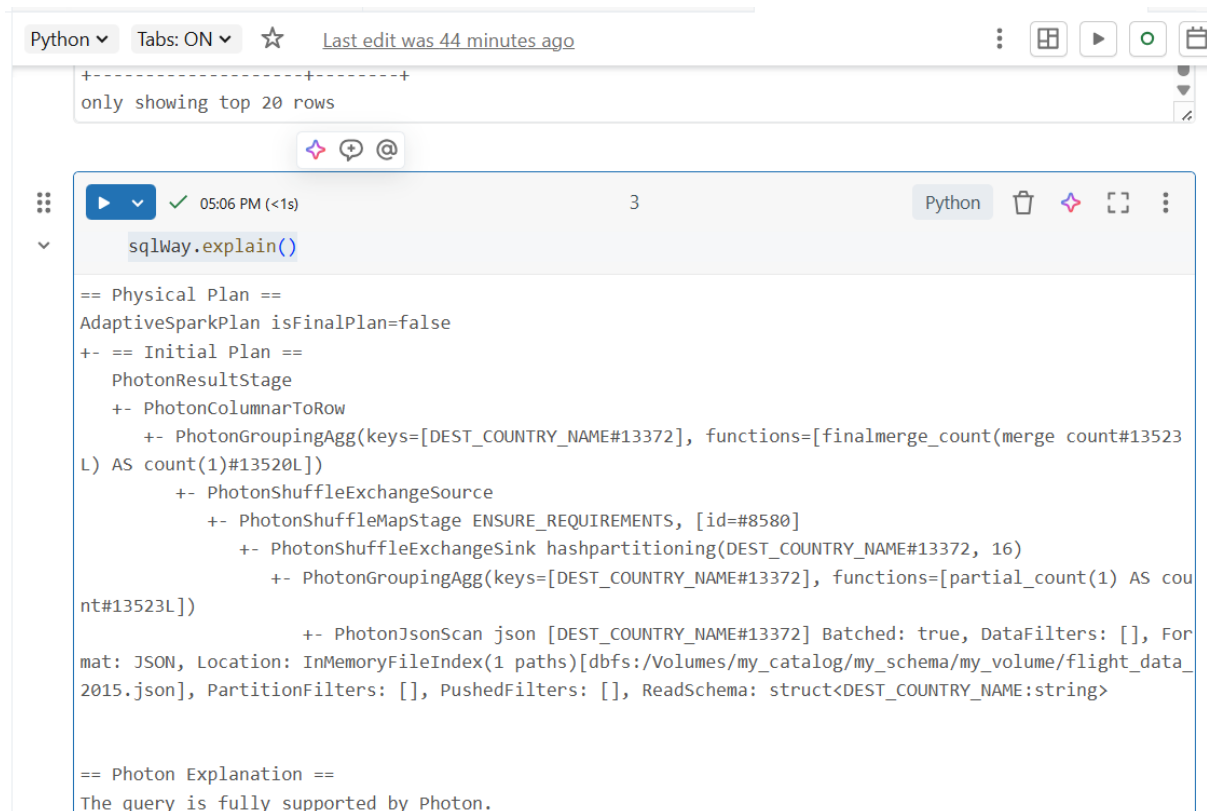


How to Read a Databricks Query Execution Plan



The screenshot shows a Databricks notebook interface. At the top, there's a header bar with 'Python' selected, 'Tabs: ON', a star icon, and 'Last edit was 44 minutes ago'. Below this, a toolbar contains icons for grid view, play, stop, and a calendar. A status bar indicates 'only showing top 20 rows'. The main code cell is titled 'sqlWay.explain()' and contains the following output:

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- == Initial Plan ==
    PhotonResultStage
      +- PhotonColumnarToRow
        +- PhotonGroupingAgg(keys=[DEST_COUNTRY_NAME#13372], functions=[finalmerge_count(merge count#13523L) AS count(1)#13520L])
          +- PhotonShuffleExchangeSource
            +- PhotonShuffleMapStage ENSURE_REQUIREMENTS, [id=#8580]
              +- PhotonShuffleExchangeSink hashpartitioning(DEST_COUNTRY_NAME#13372, 16)
                +- PhotonGroupingAgg(keys=[DEST_COUNTRY_NAME#13372], functions=[partial_count(1) AS count#13523L])
                  +- PhotonJsonScan json [DEST_COUNTRY_NAME#13372] Batched: true, DataFilters: [], Format: JSON, Location: InMemoryFileIndex(1 paths)[dbfs:/Volumes/my_catalog/my_schema/my_volume/flight_data_2015.json], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<DEST_COUNTRY_NAME:string>

== Photon Explanation ==
The query is fully supported by Photon.
```

When working with **PySpark in Databricks**, many developers run queries but rarely inspect what Spark is actually doing internally.

One of the most powerful tools available is the **execution plan**.

Using the `explain()` method in PySpark, we can see how Spark transforms a query into multiple execution stages.

Example:

```
sqlWay.explain()
```

Output (simplified):

Let's walk through the execution flow.

1. Data Scan

PhotonJsonScan

Spark begins by scanning the JSON file stored in DBFS.

Key details visible in the plan include:

- File format (JSON)
- Data location
- Columns being read
- Applied filters

Spark also performs **column pruning**, meaning only the required columns are loaded instead of the full dataset.

This reduces IO cost and improves performance.

2. Partial Aggregation

PhotonGroupingAgg (partial_count)

Spark performs an initial aggregation inside each partition.

For example, if the query is:

```
SELECT DEST_COUNTRY_NAME, COUNT(*)  
FROM flights  
GROUP BY DEST_COUNTRY_NAME
```

Each partition calculates a **partial count**.

Example:

Partition 1

Country Count

India 10

USA 5

Partition 2

Country Count

India 7

USA 8

This significantly reduces the amount of data that needs to be transferred across the cluster.

3. Shuffle Phase

PhotonShuffleExchangeSink

Spark redistributes the data across nodes based on the grouping key.

In this case:

```
hashpartitioning(DEST_COUNTRY_NAME, 16)
```

This means Spark creates **16 partitions** and ensures all rows with the same country go to the same partition.

This process is called a **shuffle**, and it is one of the most expensive operations in distributed systems.

4. Shuffle Map Stage

PhotonShuffleMapStage

This stage prepares the shuffled data so that it can be consumed by the next stage of computation.

It ensures that partitioning requirements are satisfied before aggregation continues.

5. Final Aggregation

PhotonGroupingAgg (finalmerge_count)

Spark merges the partial counts from all partitions to produce the final result.

Example:

Country	Partial1	Partial2	Final
India	10	7	17
USA	5	8	13

6. Columnar to Row Conversion

PhotonColumnarToRow

The Photon engine processes data in **columnar format** for high performance.

Before returning the results, Spark converts the data back into **row format**, which is required for DataFrame output.

7. Result Stage

PhotonResultStage

This stage sends the final result back to the user or notebook.

8. Adaptive Query Execution

AdaptiveSparkPlan

Spark uses **Adaptive Query Execution (AQE)** to optimize queries dynamically during runtime.

It can automatically adjust:

- Shuffle partitions
- Join strategies
- Data skew handling

This allows Spark to improve performance based on real runtime statistics.

9. Photon Engine

The plan also shows:

The query is fully supported by Photon

Photon is the **vectorized execution engine used by Databricks**, written in C++.

Benefits include:

- Faster aggregations
- Optimized joins
- Improved CPU utilization
- Reduced query latency

In many workloads, Photon can deliver **2x–8x performance improvements** compared to the standard Spark engine.

Final Thoughts

Reading execution plans is an essential skill for anyone working with **Spark, Databricks, or large-scale data processing**.

Understanding the internal stages helps in:

- Diagnosing performance issues
- Reducing expensive shuffles
- Optimizing aggregation logic
- Improving cluster efficiency

The next time you write a PySpark query, try running:

```
df.explain()
```

It provides a window into how Spark actually executes your data pipeline.